

Innovative Guitar System for Beginners

StrumTech

Overview

This project is an All-in-One beginner guitar. Whenever someone is interested in learning how to play an electric guitar, there is a large barrier to entry on components. Just to start, a beginner must purchase a guitar, an amp, and a cable for the guitar to connect to the amp. Even further, a tuner is usually required to correct the tuning on the strings. Then further cables are required if the user acquires analog pedals or even a digital pedalboard. All of these components can be daunting for a newcomer. Our guitar design will encompass many of the components a beginner needs (tuner, digital pedalboard, speaker/amplifier) in order to make the initial jump into playing the guitar much easier.

Team Members and Performed Tasks

David Ellis - Digital Effects, Circuitry and Guitar Body

Austin Petersen - PCB, 3D Print Designs and Guitar Body/Neck

Gerald Stanford - App, Coding and Testing

Michael Ward - Coding, Circuitry and Guitar Body/Neck



Figure 1: The Team

Problem Statement

Introduction, Background, and Other Motivations for the Problem:

All of the members of our group play the guitar at some level. Therefore, we wanted to create a project that would relate to the guitar. After discussing aspects of our guitar playing, we all had a similar experience, that of a large initial investment in a series of components to get started playing the electric guitar. When a beginner starts playing the electric guitar, they first have to buy the guitar, then a cable and amplifier for said guitar. Then, if they want to experiment with different pedals, they have to purchase either multiple individual pedals, all requiring connecting cables, or a multi-effect pedal, which reduces the amount of cables required, but still needs a connection to the wall. Also, as a beginner, you are trying to deal with all of these components and cables, while learning new information. This generally comes from a book or a website, which adds another device that needs to be present during practicing. Along with this, every guitarist needs a tuner.

Concise Problem Definition:

Because of the large barrier of entry to beginner electric guitarists, due to the large amount of devices needed to fully access the capabilities and tones of the guitar, our goal is to create a beginner guitar that encompasses all of the essential components in order to get started.

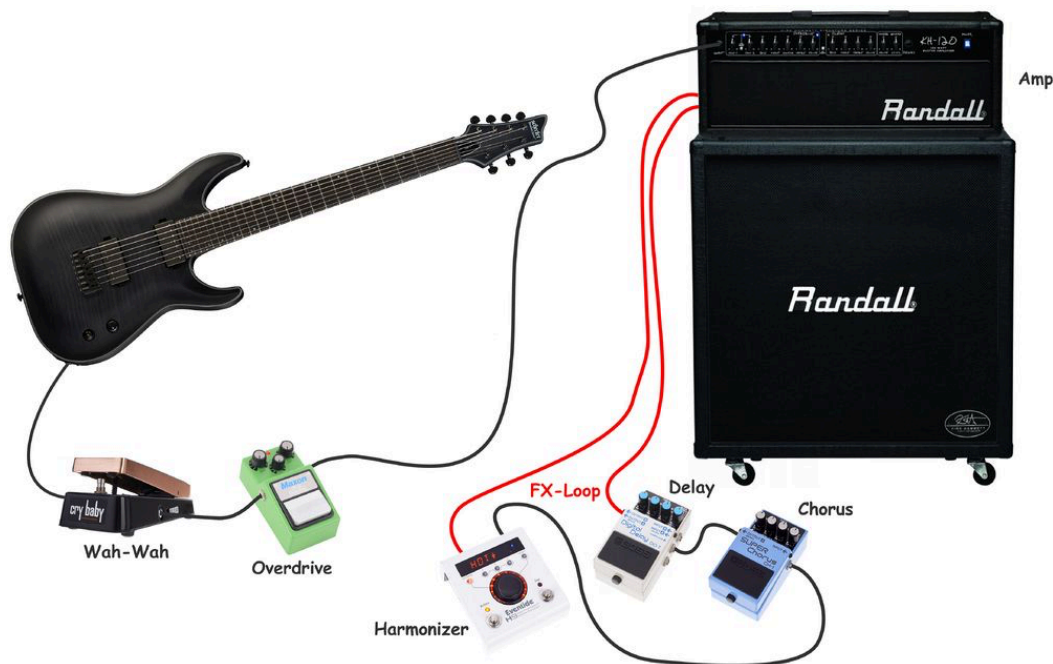


Figure 2: Example Guitar Setup that Illustrates our Problem

Objectives/Goals/Requirements

Overall:

Overall System Goals

The overall system goal is to design a complete and functional guitar and learning environment that will allow the user to enjoy the fulfillment that comes with not only playing an instrument but also learning one. However, the package is designed to accomplish two main goals. The two goals are to negate some of the financial requirements that accompany purchasing an instrument, and particularly in this case a guitar. As well as to ease the learning curve that comes with learning any new instrument. There are numerous additional accessories that are arguably required to get the most out of an electric guitar. Amplifiers to get the sound from the instrument and to make the music played by the user available for everyone to hear. A tuner, to ensure that while the user is playing, they can change their strings to the desired base note. Digital Effects to allow the player to express their creativity beyond what the base instrument allows and discover new sounds and ideas in the process. To name just a few, these are all supplementary features that any player, but especially a beginner to guitar would likely appreciate through the initial purchase.

In order to aid in the development of a new player's growth, two main subsystems are employed. One is visual aid provided to the user in the form of LEDs. These LEDs are situated on the fretboard to show the user where their fret hand should go to play the chords they intend to play. By utilizing a database of common and potential chords the user would like to play, the user query it and can be shown exactly where their hand should be placed on the fretboard and the proper finger positioning they should use. This allows the user to become familiar with different chords and chord patterns without needing to fully understand the fretboard initially or needing to consult external sources, such as lesson books or the internet, which can take additional money and or time that can be spent simply playing and practicing. The second subsystem is the Guitar Learning app, that is designed to provide the user an interface for interacting with the guitar system but also provide learning materials and necessary challenges that players can use to expedite their growth. All of these components are tied together with the onboard microcontroller that acts as both a means of communicating with the application and handling all onboard processing and component activation.

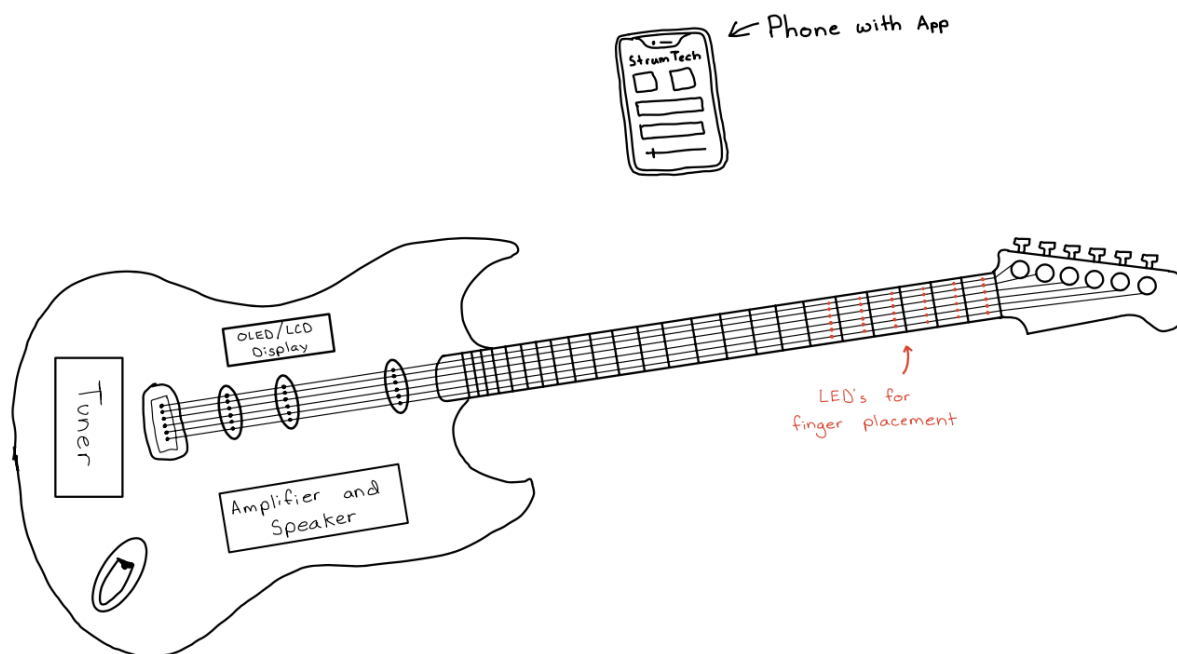


Figure 3: Initial Guitar Concept

Realistic Design Constraints:

The main constraint that is unique to our project is the limited size. Our project design is to have all of the components embedded within the guitar. Therefore, our physical components are limited by the size of the guitar. We are also limited by the price and time of our project. Our project must be completed within two semesters and be less than \$250.

A few additional constraints arose as a result of design choices made in the fall. Our choice of microcontroller and development ecosystem constrained the work that we did in the spring. Namely, we were limited to the processing power of the chosen microcontroller which needed to control the light array, the digital signal processing, the tuner, and bluetooth communication. The constraints related to the Apple development ecosystem for the mobile app were the cost of the program and the lack of testability.

Planning by Semester:

Fall Goals

For the fall semester, our goals were to mostly complete the physical components of this project. Our specific goals are as follows:

- Complete Tuner Revision
- Complete Amplifier Circuit
- Complete Signal Processing
- Complete LED Fretboard Prototype

- Begin Phone Prototype

Spring Goals

For the spring semester, our goals are to complete the app and insert the physical components into the guitar. Our specific goals are as follows:

- Complete Phone App, including
 - Tuner Interface
 - Digital Pedal Interface (if the signal processing is completed; see challenges for more details)
 - Learn, Test, and Challenge Modes (if time permits)
- Build Guitar
- Insert Components into Guitar

Prior Work

Tuner Module:

In sophomore design, Michael designed an audio frequency sensor, which will be the basis for the tuner module in this project. This sensor takes in sound and performs a fast fourier transform on the signal, which returns the frequency of the wave with the greatest amplitude and the pitch/note of that wave. This is produced into the serial monitor in the Arduino IDE every second. In order to use this design in our project, given that it was initially designed for an acoustic guitar by taking in the sound waves with a microphone, this section of the module can be scrapped and all that is needed is the code. This is because the signal produced by the guitar can be directly wired into the microcontroller, rather than relying on the microphone collecting this signal, then sending it to the microcontroller.

Pedal Building:

David has personal experience in building analog guitar pedals, using schematics from buildyourownclone.com as a basis for design. This prior experience unfortunately did not translate into success building digital guitar effects for this project. Digital guitar effects require DAC's ADC's and a lot of coding in order to replicate what analog effects do using capacitors, resistors, amplifiers, and other components.

Approach

Design of the Neck and LED Fretboard:

After looking at other designs people have done, we decided to implement our LEDs into the physical neck of the guitar. In order to do this, we had to design PCBs which could be sunk into the neck, cut channels in the neck to allow for this implementation, and provide some kind of cover over these PCBs to allow for the neck to still be playable.

During the fall semester, we designed our first iteration of the PCBs for the LED Fretboard. This PCB design consisted of six Adafruit NeoPixels in series per board, along with connections for power, ground, data in and data out. These PCBs would then be connected in a string of six to provide chord information for the first six frets of the neck. By the end of the fall semester, we had our first design for the PCBs and code which would work them.



Figure 4: Cutting the Channels

At the start of the spring semester, we sought to improve our PCB design. An error we were having with the PCBs was the interconnectivity between each board. Because the connections were surface mount, the constant contact with the user and movement of the neck resulted in many of the connections being broken and stripping off the solder mask of the board, making re-soldering difficult. Therefore, we designed new boards, changing the connection from surface mount to through-hole. Since this change, we have not experienced any breaks in connection between the boards.

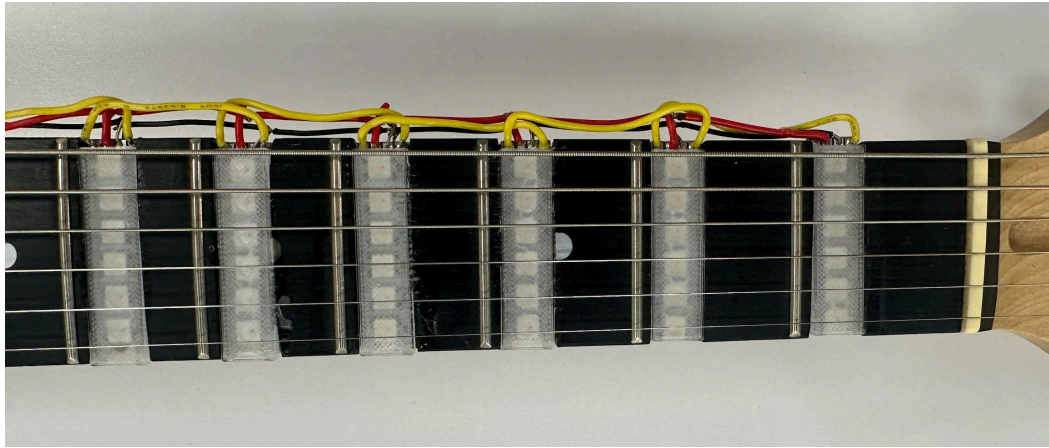


Figure 5: The Final LED Fretboard

The next step was to insert the boards into the neck. We measured channels on the fretboard to the size of our PCBs and cut these channels out. Then, we sunk the PCBs into the neck by epoxying them to the wood, then hot gluing translucent 3D printed covers onto the PCBs to level out the surface and make the fretboard playable.

Design of the Guitar Body:

After completing the circuitry for the hardware to be implemented in the guitar, we realized we did not have enough space. We could not fit all the hardware inside the wood of the body of the guitar. It was decided that we will use 3D printed compartments to house all our electronics. We looked at the body of the guitar and rough sketched straight lines where we could cut the body and add the 3D prints without messing up any parts of the actual guitar. We then measured the length of the drawn lines and went to onshape to design the compartment. Initially we thought we only needed one extra space, but again we did not have enough space. So an extra line was drawn on the guitar and the new compartment was designed. It took a few test prints at first, but then we got a design that fit well together and looked aesthetically pleasing. Printing both compartments as one could not be done because that size would not fit in the 3D printer. Once we had the finalized shape of the compartments we went to the maker gym to cut out the body of the guitar.



Figure 6: Guitar Body

After the guitar was cut and the compartments were printed it was time to figure out how secure the prints were to the guitar itself. This was done by implementing screw holes into the sides of the prints this way they could be screwed into the body of the guitar. Each piece was secured using 5 screws, each had 3 on the long flat sections and 2 screws in the shorter flat sections. Screw holes were also needed for securing our hardware in the compartments. The Esp32, amplifier circuit and buck converter were the components fastened to the 3D prints.

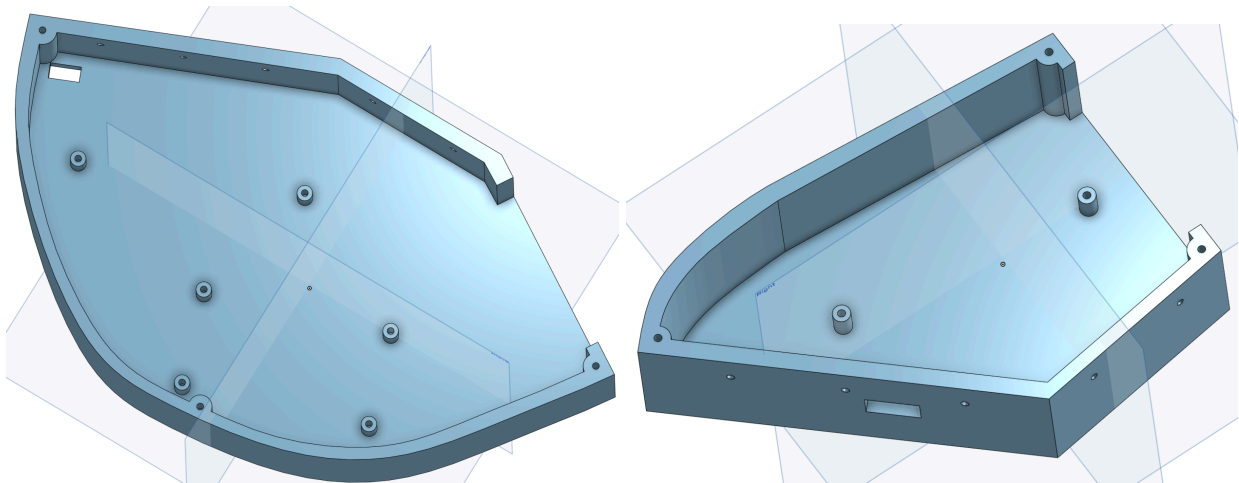


Figure 7: Compartment Cad Designs



Figure 8: Guitar Body with 3D Print Cavity

After the compartment and the guitar were done we moved to the lid next. We decided to use a clear laser cut acrylic, this way you can see the hardware and how it is connected. The acrylic also had to be cut in two separate pieces like the compartments due to size of the laser cutter. We decided to also fasten the acrylic down using screws, in three corners of each compartment screw holes were added to the top to allow the acrylic to be attached. Also, the larger acrylic needed a spot for the speaker to sit for our amplifier. A hole large enough to hold the speaker without letting it fall through was added and screws with nuts were used to secure the speaker to the lid. A few more little holes were needed for a switch and some knobs. Then all the pieces were put together with the hardware for our finished prototype.

Design of the Circuit:

During the fall semester, we used a design for an LM386 circuit to provide the audio output of the guitar. We connected the output of our test guitar, using a guitar cable, to an audio jack that connects to the amplifier. For the spring semester, we needed to implement this circuit directly into the guitar. We connect the internal wiring from the guitar to our ESP32 and Teensy Audio Board. In this case, the connection to the ESP32 allows for the operation of the tuner, reading a clean signal rather than an altered one. The other connection runs to the Teensy Audio board. The Teensy Board then applies the digital effect to the signal. Then, this signal is branched off to the onboard audio amplifier mentioned above and to a guitar jack. This branching allows for the use of the onboard amplifier or an optional connection to an external amp for greater sound quality. There is also a switch to allow for the disconnection of the

onboard amplifier so that the user doesn't have to have both the onboard and external speakers on when they desire to only play on the external one.

Besides the connection of the guitar pickups and signals to our design, we also had to interconnect the ESP32 and Teensy Audio Board to allow for the operation of the digital effects. The ESP32 receives orders from the app of which audio effect to play. In relaying to the Teensy Board, the ESP32 gives a binary output (00, 01, 10, 11) to the Teensy, which each correspond to a specific audio effect. There are also two analog inputs, however they had to be converted to potentiometers, due to the high amount of noise coming from the ESP32, which we believe to be the Pulse Width Modulation of the signal.

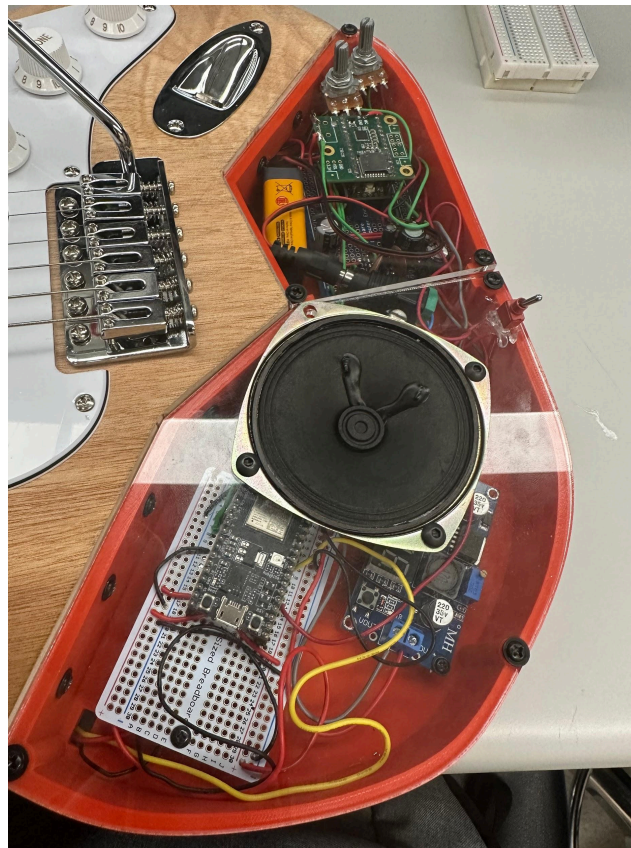


Figure 9: Circuitry and Amplifier

Design of the Microcontroller Software:

We wanted to have a system that could switch between a number of different tasks or modes for the user. Each mode had different capabilities and purpose, so a goal was to only utilize a particular part of the system when it was necessary. For example getting the tuner data was important but only when the user was expecting it. So a state machine made the most sense to implement based on the usage constraints we put in place. However because bluetooth communication with the app was an integral part of the project and controlling the guitar, the first task to accomplish was implementing the code on the controller that could establish the

controller as a bluetooth device and send data to another device. The process of writing the controller bluetooth code was fairly straightforward, testing it however required a compatible device. It was decided that the easiest approach would be to begin coding the app, so testing could occur as soon as possible. Once relatively reliable communication could be established between the two devices, designing of the state machine was next. The state machine had essentially two functions to perform, which were to determine the current mode the guitar was supposed to be in, which depended on data sent by the app and keep track of the state of that mode. Once The mode and state were established, the controller could execute certain code from libraries written by the members pertaining to that mode. As development continued more libraries were added and code was adjusted.

```

/* determines which mode to switch to and handles the switching logic*/
void ExecuteCurrentMode()
{
    Serial.println("Current Mode: ");
    Serial.println((int)CURRENT_MODE_SETTINGS);
    NEXT_MODE_SETTINGS = CheckifBluetoothRecievedModeData(CURRENT_MODE_SETTINGS);

    int nextMode = NEXT_MODE_SETTINGS;
    int currentMode = CURRENT_MODE_SETTINGS;

    if(nextMode == currentMode && currentMode != NO_MODE)
    {
        GUITAR_STATE_PROGRESS = GUITAR_STATE_CONT;
        SwitchToMode(currentMode);
    }
    else if (nextMode != currentMode && currentMode != NO_MODE && nextMode != NO_MODE)
    {
        GUITAR_STATE_PROGRESS = GUITAR_STATE_EXIT;
        SwitchToMode(currentMode);

        GUITAR_STATE_PROGRESS = GUITAR_STATE_NEW;
        NewModeInit(nextMode);
        SwitchToMode(nextMode);
    }
    else if (currentMode == NO_MODE && nextMode != NO_MODE )
    {
        GUITAR_STATE_PROGRESS = GUITAR_STATE_NEW;
        NewModeInit(nextMode);
        SwitchToMode(nextMode);
    }
    else if (nextMode == NO_MODE)
    {
        // nextMode == NO_MODE DO NOTHING just delay
    }
}

```

Figure 10: MicroController State Machine

Design of the App:

For the app, like the microcontroller code, we needed something that could communicate with the other device over bluetooth. So the beginning steps when creating the app focused around making code that could hold a connection and transmit and receive relevant data. Since the controller was created first it had the server role, while the app performed as more of a client. Once the code on both the app and controller were in a state where they could communicate with one another, that started the initial phase of debugging both devices. Once completed creating the user interface for the app became the next task. While creating the UI was simple enough, getting the different screens of the app to send data to one another and making sure each screen was responsible for ensuring that the connection between the controller was maintained took a bit of troubleshooting. So instead of sending multiple pieces of data between screens there was one common object that they all had access to that was responsible for handling bluetooth interactions. Two additional screens were created as well. One for switching between guitar modes and another for reconnecting the app to the controller. The separation of tasks on the screens made it easier to code as well as isolate problems when debugging. The entire process for writing both the app and controller code was an iterative process, making adjustments in code and data processed to better meet the needs of the design.

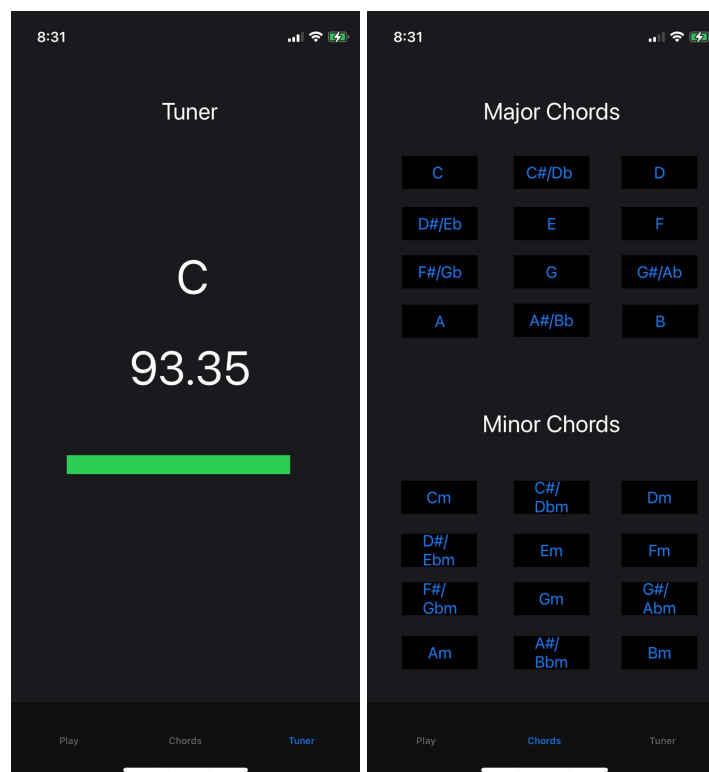


Figure 11: Tuner and Chord Library Screens

Design of Digital Effects Subsystem:

Our approach for designing the digital effects system changed this semester compared to our attempts in the fall. Originally we were trying to use one microcontroller for everything, meaning the ESP32 would control the digital effects in addition to the LED's, tuner, and bluetooth communications. This turned out to be very difficult as the ESP32 does not have a good enough ADC or DAC for high quality audio, so we would need an external ADC/DAC to communicate with the ESP32 via I2S. After tinkering with a lot of different options to solve these problems in the fall, in the spring we adopted a new strategy based on some work that has already been done by other engineers.

Our new hardware for the digital effects was the teensy 4.0 and the teensy audio shield. There is much more documentation and examples online for this system compared to using the ESP32 for digital signal processing. The teensy audio library was very useful for designing the system and choosing which effects we could use. There is an Audio System Design Tool developed by teensy where you draw a flowchart and it outputs a block of code to help with the software development. The flowchart for our effects are shown below. This chart includes stereo audio and three options for effects. There is a chorus, a feedback delay loop, and a waveshape effect on each channel. There is an I2S input and output as well as mixers that combine the effects. The mixers allow us to control which effect is active.

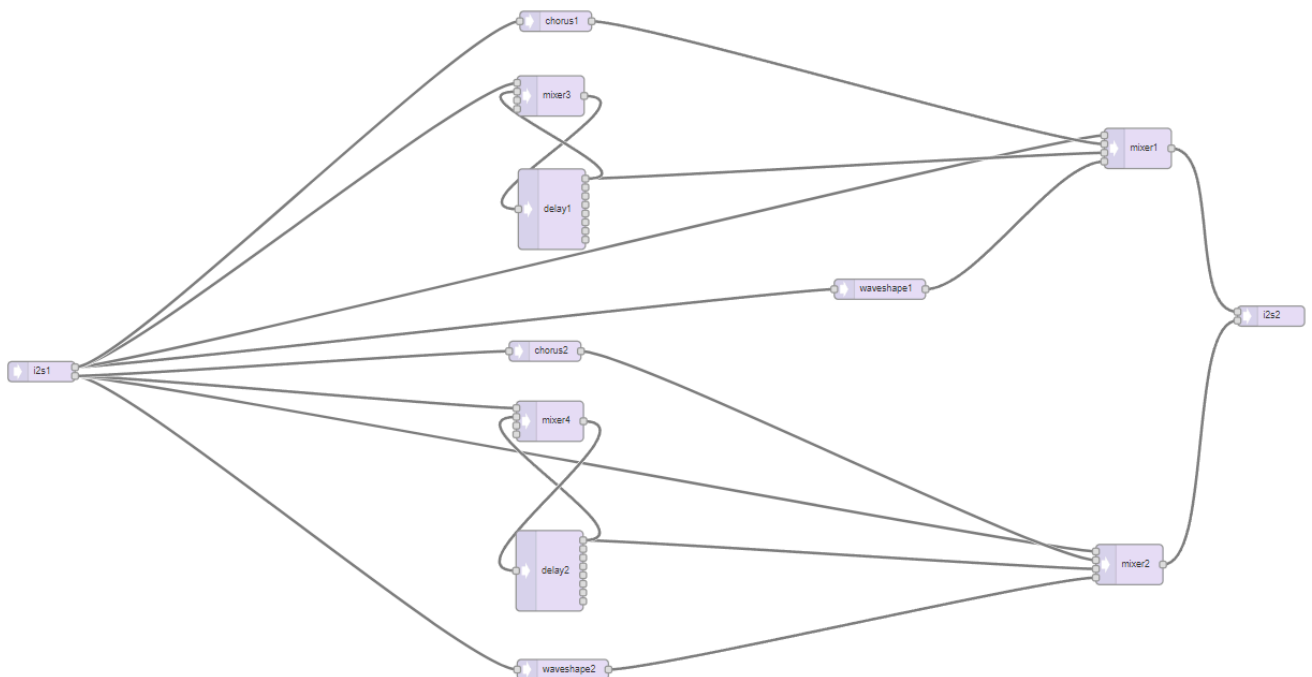


Figure 12: Digital Effects Flowchart

The hardware for the digital effects was straightforward. It involved soldering pins onto the audio shield and connecting it to the teensy. Our original design involved having the mobile

app control which effect was active and the strength of the effect. This required connecting two digital output pins and two analog output pins from the ESP32 to the teensy. The issue that occurred is that the analog output pins from the ESP32 use pulse width modulation to control the output level, which introduces noise to the system. To get around this noise we decided to use physical potentiometers to control the analog inputs while still using the mobile app to select which effect is active.

Challenges

One significant challenge due to our size constraint was the choice of which power source to use. During our initial planning phase early in the fall semester we figured we would use a rechargeable Lipo battery with a charging circuit. This would enable the user to play the guitar either plugged in or unplugged and easily charge the battery by connecting it to a wall power supply. The issue that we ran into with using a Lipo battery was space inside the guitar. Because of this constraint we shifted our plan to using only an external wall power supply with no battery inside the guitar.

Further along in project development we noticed a significant amount of unwanted noise in the output of our system. We deduced that it was coming from the wall power supply switching the AC power to DC power using a rectifier circuit. This noise was unacceptable for our project as the sound output was very unpleasant. This led us to shift one more time back to a battery, but due to the time constraint this ended up being a non-rechargeable 9V battery that would need to be replaced when it ran out of energy.

Another challenge during this project was an unexplainable error we encountered while programming our ESP32 microcontroller. The error, “Exit Status 2”, was unfixable so we had to switch to a different module for our final design.

The next challenge that we faced was attempting to hide the wires that powered the LED fretboard. Our initial design included a channel running down the length of the fretboard underneath all of the frets. The wires would run down this channel, out of sight, to the cavity which housed the microcontroller. The challenge was that we could not cut this channel without destroying the fretboard and neck and lowering the structural integrity of the guitar. This may cause it to bend or break. We decided that we would need to run the wires outside of the neck instead.

Two challenges occurred during the final stages of our project and are present in our final design. We did not have time or ability to fix these issues. The first is a feedback loop created between the built-in speaker and the guitar strings. As the speaker plays audio, it vibrates the guitar, which in turn vibrates the strings and plays more sound. This gradually increases playing louder and louder until it reaches the maximum output of the amplifier circuit. The only way to prevent this is to turn off the built-in speaker or to mute the strings with the palm of your hand as you are playing. The second challenge present in our final design is a little bit of blurring in the LED fretboard. Due to the distance between strings and the diffraction of the translucent 3D

printed fretboard pieces, adjacent LED lights can blur together making it slightly more difficult to deduce which lights are meant to be on.

Design

Overall Systems:

Our guitar consists of three different modes. First, we have the LED Fretboard. In this mode, the user can interface with the app in order to see what strings and frets they press in order to play the chord. When the chord is selected, it lights up on the app and the LEDs are turned on in the fretboard. Second, we have the digital effect mode. In this mode, the user can interface with the app in order to select a desired digital effect. When the digital effect is selected, it lights up on the app and, when playing the string and adjusting the knobs, the new sound can be heard. The third mode is the tuner. This mode shows the user the loudest current note that is being played, what frequency this note is at, and how close it is to the desired frequency through a color code of red, yellow, and green.

Our main circuitry consists of two microcontrollers and the amplifier circuit. The two microcontrollers receive signals from the guitar. The Teensy Audio Board takes the signal and applies digital effects to it. Then this signal is broken into two wires, allowing for transmission to an audio jack, to allow external play, and to the onboard speaker.

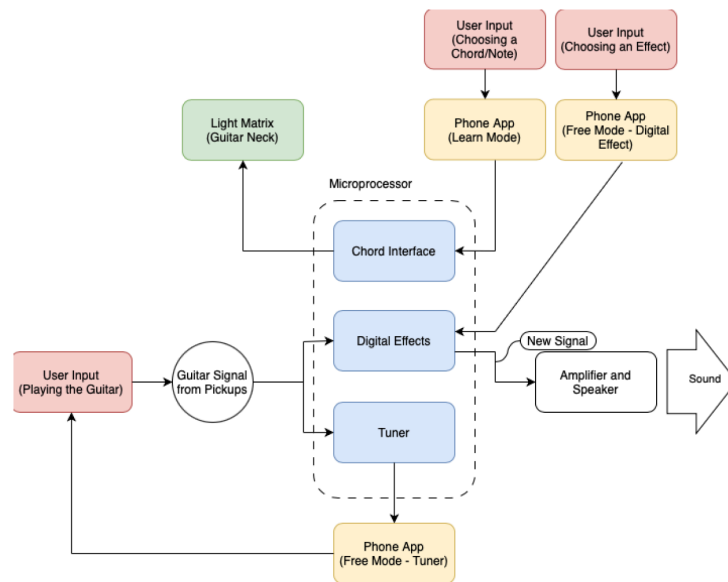


Figure 13: System Block Diagram

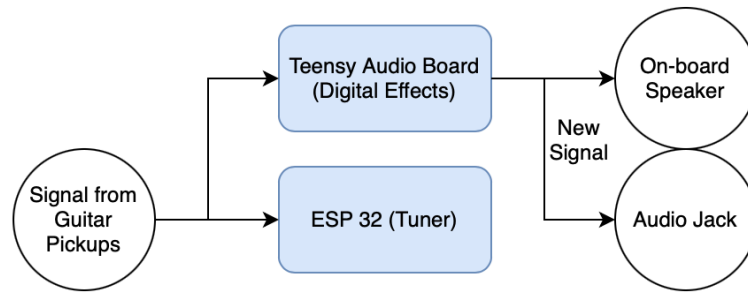


Figure 14: Audio Signal Circuit Diagram

Hardware:

We had to design two different parts for our hardware; the LED PCB Strips and the Main Circuit. The LED PCB Strips were designed using the footprint of the Adafruit NeoPixels, which allow the user to write to each LED with binary. This feature allowed us to tell each LED in the string whether to light up or remain off.

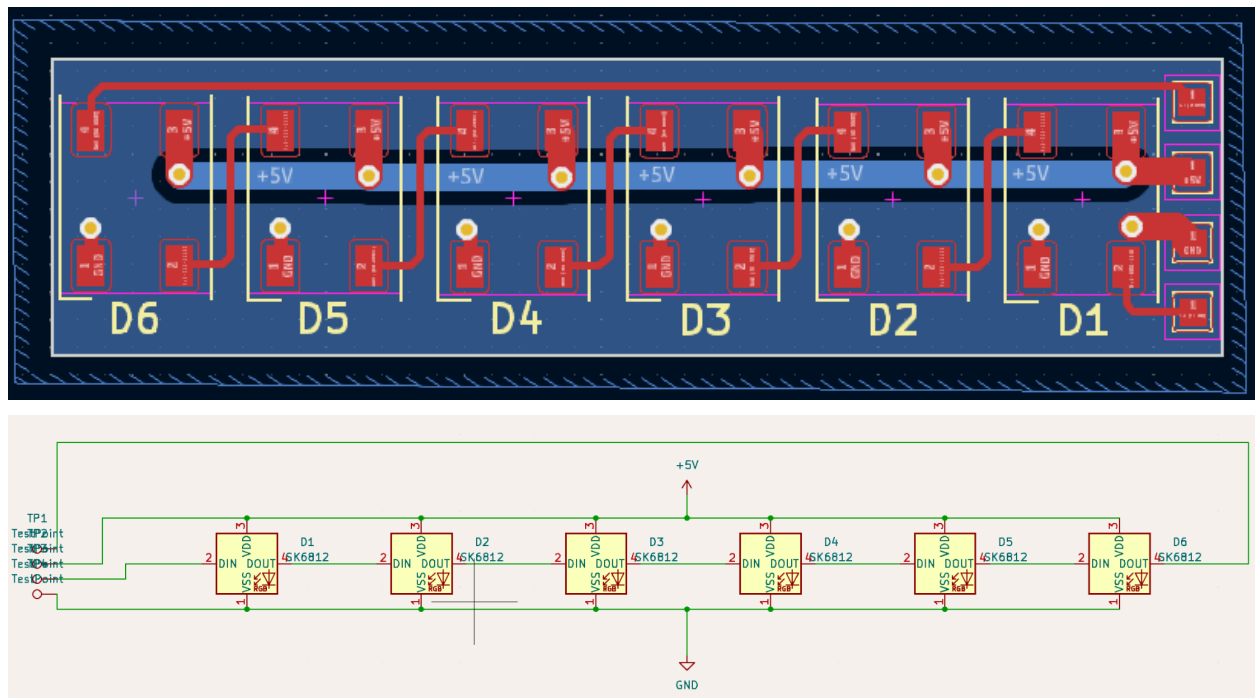


Figure 15: Initial PCB Design and Schematic

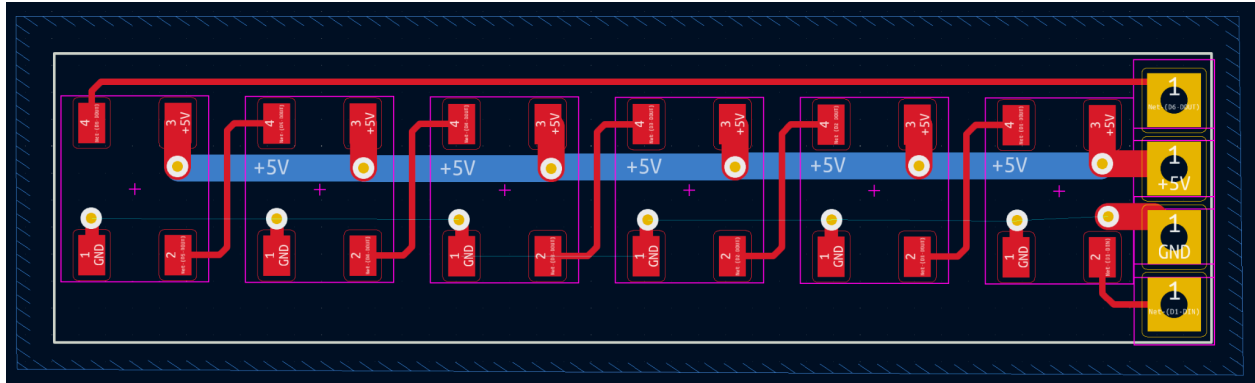


Figure 16: Final PCB Design

Our main circuitry consists of two microcontrollers and the amplifier circuit. The two microcontrollers receive signals from the guitar. The Teensy Audio Board takes the signal and applies digital effects to it. Then this signal is broken into two wires, allowing for transmission to an audio jack, to allow external play, and to the onboard speaker. There are also two potentiometers which connect to the Teensy Board, which provide an analog signal to the board to change the digital effect sent by the app.

To power this project, we use a simple 9V battery. This was decided due to the limited space available within the guitar and striving to not over complicate our circuitry and power. The 9V battery is used to power the amplifier circuit, then stepped down by a buck converter to 5V in order to power the ESP32 and the Teensy Audio Board.

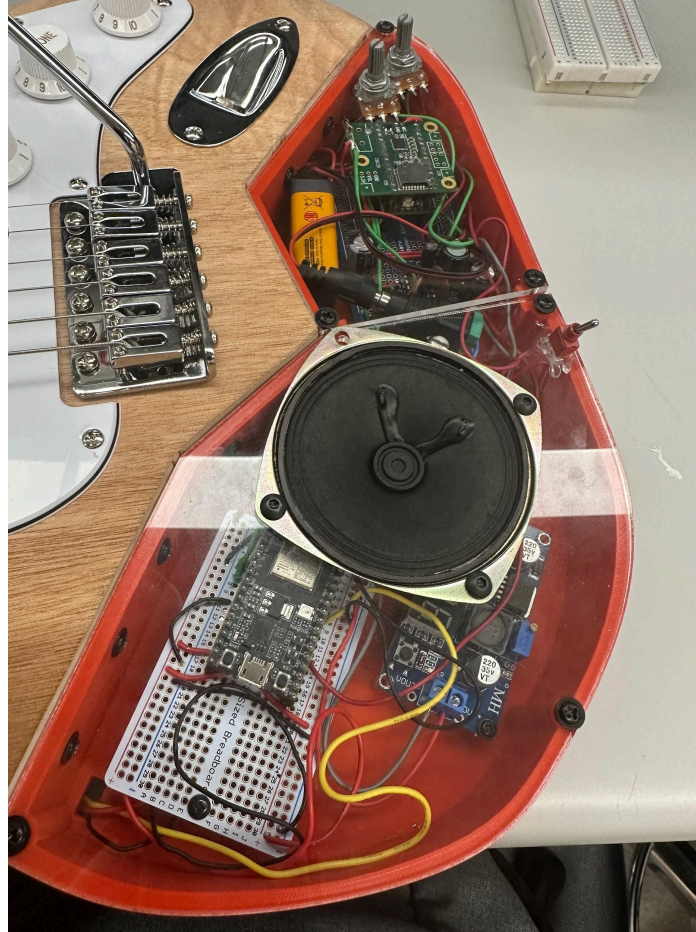


Figure 17: Circuitry of the Guitar

Software:

The software used consists of two separate libraries of code made to communicate with one another and process data sent between them. One library is run on the microcontroller, the other is designed to be installed as an iPhone app. The microcontroller consists of a state machine that calls code to transmit tuner data, light up neopixels in the neck of the guitar and communicate with the Teensy to determine the correct type and output of digital effect that should be used. The app consists of separate screens intended to transmit chord data to select the neopixels, show data to aid in tuning the guitar and select which digital effect to use.

Safety Issues

For our project, there were a few safety issues, many of which are just due to the use of various tools. To build the body of the guitar, we had to cut out space for the implementation of the 3D print casing. In order to do this, the saws at the MakerGym were used. In using the saws, safety protocol was followed. To build the neck of the guitar, we used hand saws to cut the

channels. We used safety glasses, to protect our eyes from debris. We used clamps to hold down the neck, instead of manually holding it down, which is much safer and more effective.

We had some other minor safety concerns with the actual construction of the guitar once all of the pieces were set. Firstly, working with the epoxy for the neck, we had to be careful to avoid getting it on our hands. Epoxy can cause irritation of the skin and eyes and is toxic if ingested. Working with this material, we laid down a mat of paper towels to prevent the epoxy from getting on our work table and to prevent it from getting on our hands and skin. We also used a spoon to stir the epoxy, as opposed to our hands. Secondly, though this is a minor concern, installing the strings had a slight

Project Results

Lessons Learned:

First, we learned that through-hole connections are clearly the better choice when trying to connect wires to a board. Using the surface mount connection for the wires created real difficulties in connecting the boards, and, for the fall semester presentation, made parts of the board inoperable. We had to hold one of the wires down during this presentation to get this feature to work properly.

Second, the importance of timing 3D prints was on greater display in this project. We had to go through a few iterations of our 3D printed casing. This meant that we had to spend large chunks of time waiting for the print to finish in order to test the print and to actually add it to the guitar. After about two prints, we realized that we needed to time these prints better, which led to us conducting most of our prints during the night time.

Third, we experienced the importance of protecting our circuitry from shorts. When the hole project was complete, we were testing it and all of a sudden there was a noise and the whole project stopped. Upon further analysis, the power and ground wires were shorting on one of the LED connections. To remedy this, we applied hot glue to these connections in order to insulate them, which fixed the problem .

Recommended Future Work:

There are a few things that we had set out to do in the early stages of planning our project that we removed from the plan due to time or budget constraints that could be revisited as future work for this project. In order to control the strength of the digital effects using the mobile app (instead of physical knobs) we need to implement a low pass filter to eliminate PWM noise from the analog output of the ESP32. This would most likely involve designing and manufacturing a custom PCB as well which would decrease the mess of wires that we currently have in our circuit. Due to the number of times that we had to change our design we were unable to finalize the schematic and have PCBs made, but future work would definitely involve manufacturing custom PCBs.

There are also some software developments that we would work on as future work. Firstly, we would take steps to improve the tuner code. Currently the note jumps around a lot while playing and does not reliably help the user tune the strings. Further development needs to be done to determine how to reliably detect the note that is played and how far it is from the correct pitch.

Along with the tuner improvements we want to implement chord detection. This is a similar process to note detection but it is more difficult to configure. Chord detection would allow the mobile app to display what chord is being played and provide feedback to the user if they are playing the chord correctly. The graph below, from the Department of Computer Sciences at University of Wisconsin-Madison, shows what the chord detection code has to deal with. The amplitude spectrum shows multiple peaks and taking the top three or four peaks will give you a chord.

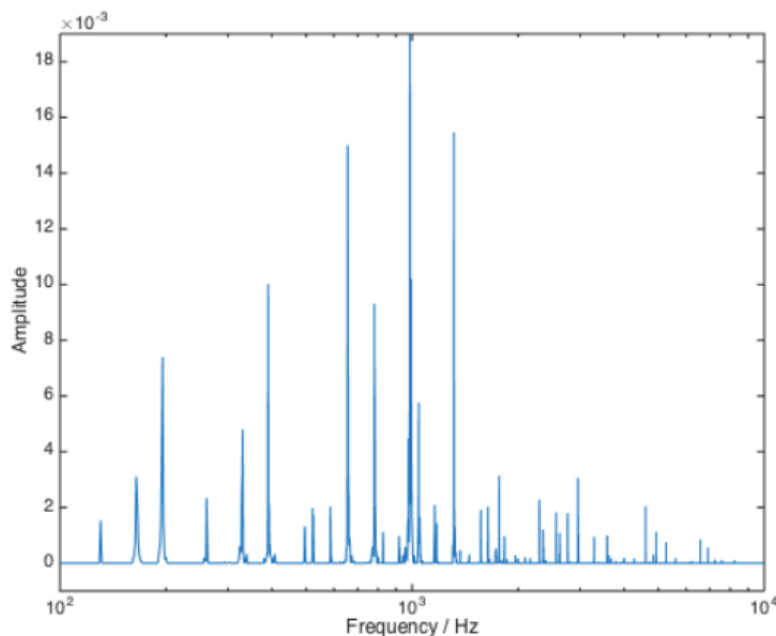


Figure 18: Amplitude Spectrum

The last item of future work that we would like to implement is a song library and AI teaching assistant within the mobile app. This would incorporate chord and note detection and the LED fretboard to teach users how to play a song and provide feedback on their playing.

Engineering Standards

ISO/IEC 14882

The ISO/IEC 14882 standard, which standardizes the C++ coding language, was used in the coding of this project. Parts of this standardization, such as removing the need for “typename” in certain circumstances and allowing for modular programming, which was

essential for building the whole program, were used. This allowed for the creation of modular functions to break apart the code and make it easier to read, understand and implement

ISO 16:1975

This standard defines the exact frequency of the note A in the treble clef. Based on this frequency we are able to extrapolate the exact frequencies of the other notes on the guitar and we can tune the guitar this way. The standard ISO 16:1975 specifies that A shall be 440 Hz with an error of no greater than 0.5 Hz. Our tuner takes into account both the frequency and the accuracy defined by the standard.

I2S Specification:

We use I2S to communicate between the microcontroller and the audio interface for our digital effects processor. I2S is a three-wire communication standard for transmitting digital sound signals, like I2C, it involves one controller and multiple targets. In our project the ESP32 will be the controller and we will have two targets: one ADC and one DAC. These will be on separate channels allowing the controller to modify the signal before outputting it.

IPC-2221

This standard covers a lot of generic rules in PCB design. I specifically followed the rules in component spacing. Components need to be spaced properly for their size and intended use. This is done to prevent a possibly short circuit and for proper component mounting and soldering. For my PCB design I only used 6 Neopixel LED components on surface mounts. Because I used these specific components they did not need to be spaced too far from each other. But for this project the LED's needed to be spaced at a specific distance to be able to fit underneath the strings of a guitar. This way each string goes over the center of each LED. I took measurements of the spacing between strings and used that as a reference on how far to space them. Luckily with this distance I was able to follow standard IPC-2221 with proper component spacing to make sure a short circuit would not happen.

IPC-2152

This standard is used for determining the copper tracing width on a PCB. Stated in the standard IPC-2152 the width of the trace must be thicker in order to sustain the amount of current traveling through it. For the PCB I designed I only had two power traces being +5 volts and ground, and two traces for data in and data out. Because the power traces have a larger current going through it, I made the trace width 1.0(mm). And because the data traces have much less current running through it than the power traces they only had to be at 0.25(mm). These sizes were both made bigger than they need to be just to be safe. With the increase in width that means there is a larger impedance in the tracing but the traces are not long in length as it is a small PCB so it doesn't have an impact on the performance of the board.

Bluetooth Low Energy (BLE)

The Bluetooth Low Energy (BLE) specification defines the standards and protocols for low-power wireless communication between devices over short distances. BLE is a subset of the Bluetooth standard and is designed for applications where low power consumption is crucial, such as wearable devices, IoT (Internet of Things) sensors, and mobile accessories. This specification is what enables the communication between the microcontroller and app to be possible.

JSON ECMA-404 Specification

JSON (JavaScript Object Notation) is a lightweight data interchange format that is easy for humans to read and write and easy for machines to parse and generate. It is based on a subset of the JavaScript Programming Language, Standard ECMA-262 3rd Edition - December 1999. JSON is a text format that is completely language-independent but uses conventions that are familiar to programmers of the C family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others. This was primarily used to send data between the two devices in situations where not only multiple pieces of data needed to be sent in a single exchange, but also multiple types, ints, strings etc. This allowed the devices to send all relevant data at once and reduced code complexity.

References

 DSP Guitar Pedal with WeAct Studio STM32H750 (Georgia Tech Capstone Design Projec...

<https://www.allaboutcircuits.com/technical-articles/introduction-to-the-i2s-interface/>

<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/i2s.html>

<https://blog.adafruit.com/2021/06/15/a-list-of-microcontrollers-with-fast-analog-to-digital-converters/>

<https://docs.arduino.cc/built-in-examples/display/RowColumnScanning>

[Core Bluetooth Overview \(apple.com\)](#)

[UIKit | Apple Developer Documentation](#)

<https://pages.cs.wisc.edu/~kma/downloads/chord-recognition.pdf>

Project Budget:

Fall	Mono Audio Amplifier	\$7.00
	Metal Shell Magnet Speaker	\$8.00
	Selector Switch	\$15.00
	GPIO Expander	\$2.00
	STM32	\$15.00
	ESP32	\$20.00
	Neopixel strip	\$6.00
Spring	Guitar Kit	\$65.00
	Audio Adapter Board	\$16.00
	External Power Supply	\$9.00
Total:		\$163.00